

ASPERITAS AI AGENT FACTORY PLAYBOOK v3.2-RC1

Internal Deployment Manual / AI Engineer Skill Stack / MVP-Performance Integration / AI-Native Company Architecture

Asperitas Inc. COO & AI Manager Office

2026-06-25

Contents

0. Truth Boundary
1. 지금까지 개발한 과정 - Chronological Playbook
2. 다음 개발부터 적용할 Upgraded Development Model
3. Core Concepts Required
4. GitHub 연결 및 운영법
5. File / Data 추가법
6. Codex / Claude / ChatGPT 사용법
7. 성능향상 방법
8. MVP Roadmap after v1.0.0-rc1
9. Deployment Playbook
10. Open Source / External Source Absorption
11. Paper / Research Documentation Layer
12. Standard Templates
13. Do Not Build Yet
14. 7-Day New Developer Onboarding
15. Final Operating Doctrine
16. 보완사항 - v3.1 Upgrade Backlog

17. 주의사항 - Do Not Break These Rules
18. 피드백 - 현재 개발 방식에 대한 냉정한 평가
19. AI-Native Company 가 되는 방법
20. AI Master 가 되기 위한 공부 로드맵
21. 자동화 방법 - Asperitas Automation Ladder
22. 다음 개발 방향 - Phase 0 to Phase 3
23. Copy-Paste Master Prompt for Next Development
24. External Reference Anchors
25. Perfect AI Engineer Skill Stack
26. MVP + Performance Integration System
27. ChatGPT / Codex / Claude 분리 운영 프로토콜
28. OpenAI / Claude / Microsoft / MIT / Stanford 학습 흡수 지도
29. AI Master 12-Week Study Roadmap
30. Automation Architecture and Implementation Ladder
31. 강화해야 할 보완점과 금지 패턴
32. v3.2 Master Prompt for Integrated AI Engineering
33. v3.2 Official Learning Reference Anchors

Reference Anchors

Executive Bottom Line

이 문서는 Asperitas 가 지금까지 진행한 AI RAG Agent 개발 과정을 내부 배포용 운영 매뉴얼로 고정하고, 다음 개발부터 OpenAI/Codex, Claude Code, Microsoft Foundry, MIT/Stanford AI 교육 체계에서 반복되는 핵심 원칙을 Asperitas 식 AI engineering 운영법으로 흡수하기 위한 표준 플레이북이다.

핵심 결론은 단순하다. 앞으로 Asperitas 의 AI 개발은 “AI 에게 코드를 맡기는 것”이 아니라 **평가 가능한 요구사항**
-> 검증 가능한 코드 -> 재현 가능한 결과 -> 추적 가능한 의사결정 -> 배포 가능한 릴리스로 움직여야 한다. 각

MVP 는 기능 추가가 아니라 하나의 실험이다. 실험에는 baseline, metric, hypothesis, implementation, verification, regression check, decision log 가 있어야 한다.

현재까지의 방향은 맞다. 이미 초안 -> ChatGPT 정리 -> Codex 구현 -> pytest/artifact/eval 검증 -> GitHub 기록 -> 릴리스 후보 확인 루프가 만들어졌다. 다음 단계의 병목은 “더 많은 기능”이 아니라 **각 MVP 마다 성능향상·워크플로우 강화·벤치마킹·자동화가 함께 붙는 개발 체계를** 만드는 것이다.

가장 중요한 운영 원칙은 네 가지다.

Source-grounded first. 출처, metadata, registry, eval 없이 에이전트를 만들지 않는다.

Eval-driven development. 느낌이 아니라 지표로 개선한다.

GitHub as memory. GitHub 는 저장소가 아니라 조직 기억장치다.

Human approval gates. 생물학·컴플라이언스·투자자·법률·외부 커뮤니케이션은 사람이 최종 승인한다.

v3.1-RC1 추가 범위:

기존 개발 루프에 대한 보완사항, 주의사항, 피드백을 별도 실행 체크리스트로 추가한다.

Asperitas 가 AI-native company 로 전환하기 위한 조직 설계, 데이터 설계, 워크플로우 설계, 거버넌스 설계를 추가한다.

AI Master 가 되기 위한 학습 로드맵을 Python/Git/LLM/RAG/Agent/Eval/Automation/Bio-AI/Compliance 까지 연결한다.

자동화 방법을 manual SOP -> checklist -> script -> CI -> scheduled workflow -> event-triggered agent -> human-approved autonomy 순서로 정리한다.

다음 개발부터 각 MVP 가 기능 개발과 동시에 성능향상, 비용/속도 측정, 보안/컴플라이언스 검증, Decision Log 업데이트를 포함하도록 강화한다.

0. Truth Boundary

이 플레이북은 지금까지의 대화, 업로드된 Asperitas AOS/Agent 문서, 내부 개발 현황 브리핑, 그리고 2026-06-25 기준 공식 문서 확인을 종합한 내부 실행 문서다.

현재 확정적으로 말할 수 있는 것:

AOS/PRIME 계열 헌법 문서와 Agent Factory 문서가 존재한다.

Source-grounded internal decision system 을 목표로 한 repository architecture, source registry, metadata, ingestion, RAG, eval, compliance gate 설계가 존재한다.

대화 기준으로 v1.0.0-rc1 내부 릴리스 후보 검증이 수행되었고, 사용자가 보고한 상태는 full pytest: 524 passed, registry_records=48, chunk_count=2821, release readiness 14/14, rc smoke 5/5, worktree clean, ready_to_tag: yes 다. 단, 이 문서 작성 세션에서는 실제 GitHub repo 에 직접 접속해 재검증한 것은 아니다.

절대 과장하면 안 되는 것:

source map 이 있다는 사실은 production DB ingestion 완료를 뜻하지 않는다.

raw source 가 있다는 사실은 legal/license review 완료를 뜻하지 않는다.

prompt 가 있다는 사실은 agent deployment 완료를 뜻하지 않는다.

RAG 설계가 있다는 사실은 vector DB/KG/eval/compliance guardrail 이 production 운영 중이라는 뜻이 아니다.

합성생물학/DBTL/실험 설계 draft 가 있다는 사실은 wet-lab validation 완료를 뜻하지 않는다.

Allowed wording:

“scaffold exists; production implementation pending”

“source registry drafted; ingestion/eval/deployment must be verified by logs”

“internal RC candidate is ready to tag based on user-provided verification report”

Forbidden wording:

“모든 DB 가 완전히 학습되었다”

“production RAG/KG/eval 이 이미 완성되었다”

“법적·규제 검토가 끝났다”

“실험적으로 검증되었다”

“foundation model for biology 가 이미 만들어졌다”

1. 지금까지 개발한 과정 - Chronological Playbook

1.1 Phase A - Strategic Constitution 구축

목표: Asperitas AI 가 일반 챗봇이 아니라 회사의 Biological Intelligence Factory 를 위한 source-grounded execution system 으로 작동하게 만드는 상위 운영 체계를 만든다.

주요 산출물:

AOS v8.0 Core Operating System

AOS v9.0 Strategic Upgrade

AOS v10.0 Source-Grounded Agent Layer

AOS v10.2 Universal Prompt & Codex Agent Constitution

AOS v10.3 Advanced Prompt Layer

ASPERITAS PRIME v2.0 Universal Master System Prompt

AGENTS.md / README.md / master_system_prompt.md

역할:

AOS v8/v9/v10: 회사 전략, biological infrastructure, source governance, compliance doctrine 정의

v10.2/v10.3: prompt architecture, output schema, private reasoning/rationale boundary, verification loop 정의

PRIME v2.0: Truth Engine, Decision Engine, Biological Intelligence Engine, AI Factory Engine, Compliance Engine 등 reusable decision engine 으로 압축

개발 판단:

이 단계는 “에이전트가 어떤 코드로 작동하는가”보다 “에이전트가 무엇을 절대 착각하면 안 되는가”를 정의한 단계다. 특히 source map 과 deployed DB, scaffold 와 production, draft 와 regulatory approval 을 분리한 점이 핵심이다.

통과 기준:

AI 가 Asperitas 미션, source hierarchy, evidence labels, compliance gate 를 이해한다.

어떤 tool 에서든 Role + Context + Task + Evidence/RAG + Constraints + Format + Verification + Iteration 구조로 지시 가능하다.

최신 헌법은 01_RAW_SOURCES/P0_ACTIVE_PROMPT/에 배치한다.

1.2 Phase B - Source Governance / Folder Architecture 설계

목표: 자료를 무작정 넣는 것이 아니라 priority, confidentiality, license, verification status, metadata 로 통제한다.

표준 폴더 구조:

asperitas-agent/
00_ADMIN/
 source_registries/
 source_manifests/
 policies/
 decision_logs/
01_RAW_SOURCES/
 P0_ACTIVE_PROMPT/
 P1_ASPERITAS_INTERNAL/
 P2_OFFICIAL_ASPERITAS_PUBLIC/
 P3_SCIENTIFIC_LITERATURE/
 P4_REGULATORY_GOVERNMENT/
 P5_INDUSTRY_MARKET_INTEL/
 P6_BENCHMARK_OPERATING_DOCTRINE/
02_REDACTED_SOURCES/
03_PROCESSED_KB/
 markdown/
 chunks/
 claim_cards/
 embeddings/
 kg_staging/
04_AGENT_APP/
 app/
 agents/
 schemas/
 tools/
05_EVALS/
 golden_sets/
 regression_reports/
 trace_reports/
06_RUNBOOKS/
07_OUTPUTS/
08_DEPLOYMENT/
99_ARCHIVE_UNVERIFIED/

Source priority:

P0 = active prompt / constitution / latest user instruction

P1 = internal Asperitas source-of-truth

P2 = approved official Asperitas public or investor material

P3 = peer-reviewed literature / scientific databases

P4 = government / regulatory / official institutional sources

P5 = industry / market / ecosystem intelligence

P6 = external benchmark doctrine

자료 추가 기본 절차:

원본 파일을 바로 처리하지 말고 raw source folder에 넣는다.

metadata.csv 또는 sources.yaml에 source_id, title, priority, confidentiality, license_status, verification_status를 기록한다.

PII, 연락처, 비공개 파트너 정보가 있으면 redacted copy를 만든다.

scripts/validate_metadata.py로 schema를 검증한다.

추출, chunking, embedding, RAG 반영은 license/confidentiality gate 통과 후 실행한다.

모든 처리 결과는 checksum과 execution log를 남긴다.

1.3 Phase C - Codex Build Guidebook / Agent Repository 설계

목표: Codex가 단순 코드 생성기가 아니라 repo-aware founding AI engineer처럼 움직이도록 build manual을 만든다.

핵심 설계:

First deliverable: repository tree, metadata schema, source priority policy, confidentiality policy, ingestion design, eval plan, first GitHub issues

Second deliverable: scripts/init_repo.py, validate_metadata.py, ingest_sources.py, chunk_sources.py, embed_chunks.py, FastAPI app, retrieval module, agent modules, eval tests, docker-compose.yml, README.md

Agent modules: COO Execution, CSO Strategy, Technical Skeptic, Species Intelligence, Genome/Protein Discovery, Pathway/DBTL Design, Literature/Patent, Compliance Gatekeeper, Grant/RFP, Market/Competitor

개발 판단:

완전 자율 wet-lab agent는 후순위다. Phase 0의 올바른 목표는 사내 문서, 전략, 규제, 연구자료를 정확히 읽고 출처를 달아 판단을 보조하는 secure RAG + agent harness다.

1.4 Phase D - Agent Factory Playbook v1/v2 구축

목표: 지금까지의 vibe coding을 회사 표준 개발 공정으로 바꾼다.

표준 루프:

초안 -> 프롬프트 -> Preflight -> 구현 -> 검증 -> 확인 -> GitHub commit/PR -> 회고 -> AGENTS.md/Skills 업데이트

역할 분담:

사용자/COO: 목표, 사업 우선순위, 리스크 허용범위, 최종 승인

ChatGPT: 전략 정리, Codex prompt 작성, 설계 리뷰, 결과 해석, 다음 MVP 분해

Codex: repository inspection, code edit, tests, verification report, PR-ready change

Claude Code: long-context repo reasoning, skills/subagents, refactor review, alternative implementation

GitHub: branch, PR, CI, diff, release, decision memory

사람: merge, release, compliance/legal/biosafety 승인

1.5 Phase E - MVP 개발 루프 정착

목표: 한 번에 완벽한 에이전트를 만드는 대신, 작은 MVP 단위로 기능과 검증을 누적한다.

MVP 단위의 최소 패키지:

MVP Brief

- objective
- user/job-to-be-done
- current behavior
- target behavior
- non-goals
- affected files
- data/schema impact
- test plan
- eval plan
- risk/compliance gate
- definition of done

현재까지 잘 된 점:

기능을 작게 나누고 검증 후 넘어가는 흐름이 생겼다.

pytest, artifact verification, registry/chunk count, smoke test, git diff clean 확인을 사용했다.

release readiness checklist 와 RC smoke 가 도입되었다.

main SHA 와 release command 를 명시하는 수준까지 왔다.

현재 부족한 점:

각 MVP 마다 “성능향상 단계”가 독립적으로 붙어 있지 않았다.

retrieval metric, answer quality metric, latency/cost metric, hallucination rate, citation precision 같은 지표가 MVP 별로 고정되지 않았다.

eval dataset 이 충분히 성장하지 않았고, benchmark source 를 코드/skill/workflow 에 구조적으로 흡수하는 단계가 약했다.

새 채팅/새 Codex 세션으로 넘어갈 때 handoff protocol 이 더 압축될 필요가 있다.

1.6 Phase F - v1.0.0-rc1 Internal Release Candidate

목표: main branch 를 내부 배포 가능한 릴리스 후보로 고정한다.

사용자 제공 검증 상태:

Main SHA: ad13324

full pytest: 524 passed

verify_artifacts: ok, registry_records=48, chunk_count=2821

release readiness: ready_for_internal_rc, 14/14 checks passed

rc smoke: passed, 5/5 checks passed

chat dry-run: dry_run_ready

security help: pass

git diff check: pass

worktree: clean, ## main...origin/main

ready_to_tag: yes

ready_for_internal_deploy: yes

Human-approved release commands:

```
git tag -a v1.0.0-rc1 -m "Asperitas AI RAG Agent v1.0.0-rc1 internal release candidate"
```

```
git push origin v1.0.0-rc1
```

```
gh release create v1.0.0-rc1 \
```

```
--title "v1.0.0-rc1 internal RC" \
```

```
--notes-file docs/releases/V1_0_0_RC1_RELEASE_NOTES.md \
```

```
--prerelease
```

주의:

이 단계는 “내부 RC 후보”이지 외부 상용 production release 가 아니다. 운영 로그, deployment environment, access control, secrets management, monitoring, backup/rollback 까지 연결되어야 production 이라고 부를 수 있다.

2. 다음 개발부터 적용할 Upgraded Development Model

2.1 Research-Lab Grade Development Loop

OpenAI/Codex 및 Claude Code 계열의 강점은 “한 번에 잘 프롬프트 쓰기”가 아니라 agent loop, tool use, context management, evals, traces, skills/subagents, CI/CD 를 합친 반복 시스템이다. Asperitas 는 다음부터 아래 루프를 표준으로 고정한다.

1. Problem Framing
2. Baseline Measurement
3. Hypothesis
4. Smallest Safe Implementation
5. Unit/Integration Tests
6. Artifact Verification
7. Retrieval/Agent Eval
8. Trace Review
9. Security/Compliance Gate
10. Human Review
11. PR / Merge / Release
12. Retrospective
13. Skill/AGENTS.md/Workflow Update

각 MVP 는 반드시 아래 질문을 통과해야 한다.

Baseline: 지금 성능은 얼마인가?

Target: 이번 변경으로 무엇이 개선되어야 하는가?

Metric: 어떤 숫자로 확인할 것인가?

Regression: 무엇이 나빠지면 실패인가?

Scope: 바꿀 파일은 어디까지인가?

Safety: biosafety/compliance/security risk 는 있는가?

Deployment: 내부 사용자에게 어떻게 전달되는가?

2.2 MVP 마다 Performance Pack 을 붙인다

앞으로 “MVP 완료”의 정의를 바꾼다. 기능만 되면 완료가 아니라, 아래 Performance Pack 까지 붙어야 완료다.

MVP Performance Pack

- metric_name
- baseline_value
- target_value
- test_dataset
- eval_script
- before_result
- after_result
- regression_check
- latency/cost impact
- failure cases found
- decision: ship / iterate / rollback

RAG/검색 MVP 의 대표 지표:

context recall

context precision

source priority accuracy

citation exactness

hallucination rate

abstention correctness

Korean COO-style actionability

compliance trigger recall

retrieval latency

token cost per answer

Agent workflow MVP 의 대표 지표:

correct tool selection

correct handoff

schema validity

task completion rate

human review burden

failure recovery quality

unsafe action refusal

log completeness

2.3 Benchmark-Driven Upgrade

벤치마킹은 “좋아 보이는 자료를 읽는 것”이 아니라 code/process asset 으로 흡수되어야 한다.

벤치마킹 흡수 프로토콜:

Scout -> Source Verify -> License/Terms Check -> Security Check -> Benchmark -> Adapt -> Eval -> Ledger -> Skill/Workflow Update

예시:

OpenAI Codex 방식에서 가져올 것: agent loop, sandbox, repo guidance, PR review, context 관리, eval-driven iteration

Claude Code 방식에서 가져올 것: Skills, subagents, MCP, hooks, long-context codebase operation

GitHub 방식에서 가져올 것: protected main, required status checks, dependency review, SBOM, release notes, PR templates

Ginkgo/OpenAI autonomous lab 논문에서 가져올 것: schema-validated experiment specification, human loading/unloading boundary, iterative hypothesis generation, cost/titer objective optimization. 단, Asperitas MVP 에서는 wet-lab execution 으로 바로 가지 않고 protocol registry/LIMS/ELN-ready data model 부터 만든다.

3. Core Concepts Required

3.1 RAG

RAG 는 모델이 문서를 “기억했다”고 주장하는 방식이 아니다. 사용자 질문이 들어오면 source registry 와 processed KB 에서 관련 chunk 를 검색하고, 그 근거로 답변을 생성하는 구조다.

Asperitas RAG 의 필수 구성:

source registry -> metadata schema -> raw source -> redaction/license review -> extraction -> semantic chunking -> embeddings -> vector DB -> retrieval -> reranking -> citation -> answer -> eval -> decision log

3.2 Source Registry

모든 source 는 knowledge object 가 아니라 governed asset 이다. 최소 metadata:

source_id, title, file_name, source_type, priority, origin, author, date,
last_verified, confidentiality, license_status, use_permission, reliability,
jurisdiction, tags, entities, linked_projects, evidence_level,
verification_status, risk_notes, action_items, checksum

3.3 Chunking

청킹은 단순히 문서를 1000 자씩 자르는 작업이 아니다. RAG 의 성능은 chunk boundary 에 크게 좌우된다.

권장 방식:

heading/section-aware chunking

slide/page provenance 유지

table/figure caption 별도 보존

chunk 마다 source_id, page/slide, priority, confidentiality 부착

과도하게 긴 chunk 와 너무 짧은 chunk 를 모두 피한다

chunking 변경 시 반드시 retrieval eval before/after 비교

3.4 Embedding / Vector DB / Hybrid Search

Phase 0 은 local or controlled PostgreSQL + pgvector 를 기본값으로 한다. 나중에 필요하면 managed vector DB 를 검토한다.

권장 retrieval stack:

BM25 keyword search + vector similarity + source-priority weighting + metadata filter + reranker + citation checker

3.5 Knowledge Graph

KG 는 Phase 0 필수가 아니라 Phase 2/3 확장이다. 다만 schema 는 미리 KG-ready 로 설계한다.

초기 entity:

species

gene

protein

pathway

compound

experiment

sample

source

claim

regulation

partner

product

patent

3.6 Evals

Eval 은 나중에 하는 품질검사가 아니라 개발의 중심이다. 특히 RAG 와 agent 는 “작동하는 것처럼 보이는” 경우가 많기 때문에 golden questions, adversarial cases, regression cases 가 필요하다.

Eval 종류:

source grounding eval

citation exactness eval

retrieval recall/precision eval

compliance trigger eval

hallucination/unsupported claim eval

Korean executive style eval

actionability eval

latency/cost eval

trace grading for agent workflows

3.7 Traces

Trace 는 agent run 의 블랙박스를 열어주는 실행 기록이다.

기록해야 할 것:

user task

selected agent/tool

retrieved sources

tool calls

model outputs

guardrail decisions

final answer

human review decision

failure mode

3.8 Guardrails

Asperitas 의 guardrail 은 단순 profanity filter 가 아니다. 아래 action 을 막거나 human approval 로 올리는 구조다.

CITES/Nagoya/LMO/biosafety/biosecurity issue

wet-lab protocol execution

genetic engineering action

sample transfer/import/export decision

legal/regulatory conclusion

investor claim

patent/FTO conclusion

external communication

confidential source disclosure

3.9 Agent Harness

Agent harness 는 모델 하나가 아니라 routing, tools, memory, schemas, evals, guardrails, logs 를 묶은 실행 환경이다.

Phase 0 harness:

FastAPI app

retrieval.py

router.py

agents/*.py
schemas/*.py
guardrails/*.py
evals/*.py
logs/run_log.jsonl
decision_log.md

3.10 Skills / AGENTS.md / Prompt Library

반복되는 지시는 채팅에 계속 붙이지 않는다. 절차가 생기면 skill 로 승격한다.

승격 기준:

같은 prompt 를 3 번 이상 반복한다.

체크리스트가 길어진다.

실수 방지 규칙이 중요하다.

특정 파일 구조나 command 가 반복된다.

신규 개발자가 따라 해야 한다.

4. GitHub 연결 및 운영법

4.1 GitHub 기본 원칙

GitHub 는 코드 저장소가 아니라 Agent Factory 의 조직 기억장치다. code, test, eval, PR discussion, decision log, release notes 가 모두 다음 agent 작업의 context 가 된다.

기본 규칙:

핵심 repo 는 private.

main 은 항상 안정판.

직접 main push 금지.

모든 변경은 feature branch -> PR -> CI -> review -> merge.

CODEOWNERS 로 retrieval, ingestion, compliance, infra 책임자를 지정한다.

release 는 tag 와 GitHub release 로 남긴다.

4.2 GitHub Desktop 기준 절차

GitHub Desktop 에서 repo clone.

Fetch origin 으로 최신 상태 확인.

새 branch 생성: feature/mvp-016-retrieval-eval-upgrade.

Codex/Cursor/Claude Code 가 수정한 파일을 확인.

좌측 changed files 에서 예상 외 파일이 있는지 확인.

각 diff 를 보고 scope creep, 대량 포매팅, 삭제를 확인.

Summary 는 conventional commit 형식으로 작성.

Description 에는 tests run, eval result, risk 를 짧게 작성.

Commit to branch.

Push origin.

GitHub 웹에서 PR 생성.

PR template 작성.

CI/review 통과 후 merge.

local main pull.

4.3 Git CLI 기준 절차

first setup

```
git clone git@github.com:<ORG>/<REPO>.git
```

```
cd <REPO>
```

```
git status
```

new task

```
git checkout main
```

```
git pull origin main
```

```
git checkout -b feature/mvp-016-retrieval-eval-upgrade
```

after edits

```
git status
```

```
git diff --stat
```

```
git diff
```

```
python -m pytest
```

```
python scripts/verify_artifacts.py
python scripts/run_retrieval_eval.py # if retrieval/chunking/scoring changed
```

```
git add <changed-files>
git commit -m "feat(eval): add retrieval regression pack for MVP-016"
git push -u origin feature/mvp-016-retrieval-eval-upgrade
```

4.4 GitHub CLI PR / Release

auth

```
gh auth login
```

create PR

```
gh pr create \
  --base main \
  --head feature/mvp-016-retrieval-eval-upgrade \
  --title "feat(eval): MVP-016 retrieval regression pack" \
  --body-file docs/pr/MVP_016_PR_BODY.md
```

check CI

```
gh pr checks
```

merge after approval

```
gh pr merge --squash --delete-branch
```

release candidate tag

```
git checkout main
git pull origin main
git tag -a v1.0.0-rc1 -m "Asperitas AI RAG Agent v1.0.0-rc1 internal release candidate"
git push origin v1.0.0-rc1
```

```
gh release create v1.0.0-rc1 \
  --title "v1.0.0-rc1 internal RC" \
  --notes-file docs/releases/V1_0_0_RC1_RELEASE_NOTES.md \
  --prerelease
```

4.5 Branch Protection 설정

GitHub repo settings 에서 main 에 다음을 적용한다.

Require a pull request before merging

Require approvals

Dismiss stale approvals when new commits are pushed

Require status checks to pass before merging

Require branches to be up to date before merging

Block force pushes

Block deletions

Require conversation resolution before merging

CODEOWNERS review required for critical modules

Required checks:

pytest

artifact_verify

retrieval_eval

security_scan

dependency_review

lint/typecheck, if configured

4.6 Codex GitHub 연결

Codex web/cloud 사용 시:

Codex 에 접속한다.

GitHub account 를 연결한다.

작업할 private repo 접근을 허용한다.

repo root 에 AGENTS.md 를 둔다.

Codex 가 branch/PR 기반으로 작업하게 한다.

review 는 @codex review 또는 자동 review 설정으로 수행한다.

Codex 가 만든 PR 은 반드시 사람이 diff 와 test/eval 결과를 확인한다.

AGENTS.md 최소 내용:

Project: Asperitas AI RAG Agent

Mission: source-grounded, auditable, compliance-aware internal AI decision system.

Rules:

- Do not delete files unless explicitly requested.
 - Make the smallest safe change.
 - Preserve backward compatibility.
 - Run pytest and artifact verification.
 - Run retrieval eval if retrieval/chunking/scoring is affected.
 - Keep provenance, source priority, confidentiality, license status.
 - Do not claim production deployment unless verified by logs.
 - Report objective, files changed, tests, eval before/after, risks.
-

5. File / Data 추가법

5.1 원칙

파일 추가는 “업로드”가 아니라 ingestion pipeline 의 시작이다. raw file, metadata row, license/confidentiality decision, extracted markdown, chunk, embedding, eval 까지 연결되어야 한다.

5.2 새 PDF/DOCX/PPTX 추가 절차

Step 1. 파일 위치 결정

- AOS/agent constitution -> 01_RAW_SOURCES/P0_ACTIVE_PROMPT/
- 내부 회사 문서 -> 01_RAW_SOURCES/P1_ASPIRITAS_INTERNAL/
- 승인된 외부공개 자료 -> 01_RAW_SOURCES/P2_OFFICIAL_ASPIRITAS_PUBLIC/
- 논문 -> 01_RAW_SOURCES/P3_SCIENTIFIC_LITERATURE/
- 규제/정부자료 -> 01_RAW_SOURCES/P4_REGULATORY_GOVERNMENT/
- 산업/시장자료 -> 01_RAW_SOURCES/P5_INDUSTRY_MARKET_INTEL/
- 벤치마킹 doctrine -> 01_RAW_SOURCES/P6_BENCHMARK_OPERATING_DOCTRINE/

Step 2. metadata.csv 업데이트

Step 3. checksum 생성

Step 4. license/confidentiality 검토

Step 5. redaction 필요 여부 결정

Step 6. extraction to markdown

Step 7. semantic chunking

Step 8. embedding/vector upsert

Step 9. eval run

Step 10. decision log 기록

5.3 metadata.csv 예시

source_id,title,file_name,source_type,priority,confidentiality,date,origin,license_status,reliability,tags,verification_status,checksum,allowed_use
ASP-AOS-010,Asperitas Operating System
v10.0,Asperitas_AOS_v10_0_Prompt.pdf,internal,P0,confidential,2026-06-09,internal,pending,high,"agent;constitution;RAG",verified,<sha256>,internal_answer|RAG_only
ASP-IR-001,Asperitas IR Deck,Asperitas Inc. IR deck.pdf,internal,P1,confidential,2026-06-06,internal,pending,high,"IR;fundraising;strategy",needs_claim_approval,<sha256>,internal_answer|investor_draft

5.4 파일명 규칙

권장:

YYYYMMDD__priority__domain__short_title__version.ext
20260625__P3__protein_design__nature_de_novo_design_review__v1.pdf
20260625__P4__regulatory__nagoya_protocol_korea_guidance__v1.pdf
20260625__P1__strategy__asperitas_ir_claims_registry__v1.xlsx

금지:

최종최종.pdf
자료.pdf
새 문서.docx
download (3).pdf

5.5 ZIP 자료 처리

ZIP 은 바로 ingestion 하지 않는다.

1. 99_ARCHIVE_UNVERIFIED/zip_staging/ 에 저장
2. 압축 해제 전 파일 목록만 확인
3. 악성 파일/실행 파일 여부 확인
4. 내부 파일을 priority 별 폴더로 분류
5. metadata row 생성
6. license/confidentiality 검토 후 extraction

PowerShell 예시:

압축 풀기

```
Expand-Archive -Path .\source_package.zip -DestinationPath .\99_ARCHIVE_UNVERIFIED\zip_staging\source_package
```

파일 목록 보기

6. Codex / Claude / ChatGPT 사용법

6.1 언제 새 채팅을 열어야 하는가

새 채팅 필요:

새로운 MVP 로 넘어간다.

repo 상태가 크게 바뀌었다.

이전 대화가 너무 길어져 context pollution 이 생긴다.

설계/전략 논의와 구현 지시가 섞여 혼란스러워졌다.

release/debug/performance audit 처럼 목적이 다르다.

새 채팅 불필요:

같은 MVP 내에서 작은 수정이다.

이전 Codex report 를 이어 검증한다.

단순 command 확인이다.

새 채팅 시작 템플릿:

Context:

We are developing the private Asperitas AI RAG Agent.

Current state:

- Branch: [branch]
- Last verified SHA: [sha]
- Current MVP: [MVP-xxx]
- Tests last run: [pytest/artifact/eval status]
- Relevant docs: AGENTS.md, README.md, docs/releases/..., docs/mvp/...

Task:

[One-sentence objective]

Constraints:

- Use AGENTS.md.
- Inspect before editing.
- Smallest safe change.
- Preserve backward compatibility.
- Run pytest/artifact/eval as applicable.
- Report files changed, tests, eval before/after, risks.

Stop rule:

If scope is too large or repo state is unclear, stop and ask for approval before editing.

6.2 Codex Preflight Prompt

Preflight only. Do not edit files yet.

Inspect the repository and report:

1. current branch / HEAD / git status
2. files relevant to this task
3. current behavior and likely root cause
4. proposed smallest safe change
5. tests and verification commands to run
6. risk, unknowns, and whether the task should be split

Use AGENTS.md and existing docs. Do not over-scope.

6.3 Codex Implementation Prompt

Proceed with the smallest safe implementation based on the approved preflight.

Requirements:

- Preserve existing behavior unless explicitly changing it.
- Add or update tests for the changed behavior.
- Update docs only if behavior or commands changed.
- Run `python -m pytest`.
- Run `scripts/verify_artifacts.py`.
- Run retrieval eval if retrieval/chunking/scoring changed.
- Do not commit.

Report:

1. objective
2. files changed
3. behavior changed

4. tests run and result
5. eval before/after, if applicable
6. git diff summary
7. risks / follow-ups

6.4 Claude Code 병행 전략

Claude Code 는 다음에 강하다.

repo 전체 구조 이해

long-context refactor planning

skills/subagents 설계

complex debugging

alternative architecture review

documentation compression

Claude Code 사용 패턴:

- Explore/Plan: 대형 변경 전 구조 파악
- Skill: 반복 절차를 /skill-name 으로 패키징
- Subagent: retrieval reviewer, security reviewer, docs reviewer 등 역할 분리
- MCP: GitHub, issue tracker, local docs, monitoring dashboard 연결
- Hooks: edit 후 format/test/security check 자동 실행

6.5 ChatGPT 역할

ChatGPT 는 개발 지휘센터다.

사용자 rough idea 를 Codex prompt 로 압축

MVP scope 와 non-goals 설정

결과 보고서 해석

성능향상 방법 설계

benchmark source 를 operating principle 로 변환

release readiness 판단

논문용 개발 과정 정리

7. 성능향상 방법

7.1 성능향상은 5 종류로 나눈다

Retrieval Quality: 필요한 근거를 더 잘 찾는가?

Answer Quality: 더 정확하고 실행 가능한 답을 만드는가?

Workflow Quality: 더 적은 인간 개입으로 완료되는가?

Safety/Compliance Quality: 위험을 더 잘 감지하고 차단하는가?

Operational Quality: 더 빠르고 싸고 안정적으로 배포되는가?

7.2 Retrieval 개선

개선 아이디어:

heading-aware chunking

overlap tuning

table/figure 별도 chunk

source priority weighting

confidentiality filter

hybrid search

reranking

query rewrite

entity expansion

citations exactness check

필수 eval:

- Top-k context recall
- citation precision
- priority correctness
- no-confidential-leak test
- no-unsupported-claim test

7.3 Answer 개선

개선 아이디어:

output schema 고정

evidence label 강제

fact/inference/speculation 분리

confidence/missing data 표시

stop rule 적용

decision engine 적용: Execute / Verify / Pivot / Kill

Eval criteria:

- factuality
- source grounding
- Korean executive clarity
- actionability
- risk surfacing
- no overclaiming

7.4 Agent Workflow 개선

개선 아이디어:

router 분리

tool schema 강화

Pydantic validation

trace logging

handoff rules

retry/rollback rules

human approval queue

Eval criteria:

- correct routing
- correct tool use
- schema validity
- guardrail trigger
- task completion
- recovery from missing data

7.5 Cost / Latency 개선

개선 아이디어:

context compression

source prefiltering

smaller model for classification

larger reasoning model only for synthesis

caching embeddings

caching retrieval results

batch evals

async ingestion

Metric:

answer_latency_p50

answer_latency_p95

tokens_per_answer

cost_per_answer

retrieval_latency

embedding_cost_per_doc

7.6 Safety / Compliance 개선

개선 아이디어:

CITES/Nagoya/LMO keyword trigger

sensitive-source classifier

disclosure mode filter

external-output checker

investor-claim approval checker

wet-lab/protocol risk classifier

legal/regulatory draft watermark

Required behavior:

If task touches legal, regulatory, biosafety, biosecurity, import/export, wet-lab execution, investor commitments, patent filing, or external communication:

- classify risk
 - cite official source if applicable
 - draft only
 - require human approval
-

8. MVP Roadmap after v1.0.0-rc1

8.1 Immediate RC Tasks

- RC-1. Tag v1.0.0-rc1
- RC-2. Create GitHub prerelease
- RC-3. Run install-from-release smoke test
- RC-4. Write internal deployment note
- RC-5. Create rollback instruction
- RC-6. Freeze main except hotfix

8.2 MVP-016: Retrieval Performance Pack

Goal: RAG 검색 품질을 정량 개선한다.

Tasks:

Create golden retrieval questions.

Measure baseline Top-3/Top-5 context recall.

Add source-priority weighted scoring.

Add citation exactness checker.

Add no-confidential-leak test.

Report before/after.

Definition of Done:

pytest pass

artifact verify pass

retrieval eval before/after report exists

no regression in existing pass/fail behavior

PR contains metrics and failure cases

8.3 MVP-017: Compliance Gatekeeper MVP

Goal: CITES/Nagoya/LMO/biosafety/legal/external-output triggers 를 자동 감지한다.

Tasks:

Create risk taxonomy.

Create trigger keyword + semantic classifier.

Add official-source-required flag.

Add human approval output mode.

Add eval cases for false negatives.

Definition of Done:

high-risk queries never produce final action without approval

regulatory/legal outputs marked draft-only

eval false negative rate acceptable and reviewed

8.4 MVP-018: Agent Trace / Decision Log

Goal: 모든 agent run 을 나중에 재현 가능하게 기록한다.

Tasks:

Create run_log.jsonl schema.

Log retrieved sources, tools, outputs, guardrails, decision.

Add trace viewer or markdown export.

Add privacy redaction for logs.

Definition of Done:

each response has run_id

trace includes source ids and guardrail decisions

logs exclude raw confidential content unless local/internal only

8.5 MVP-019: Skills / Workflow Pipeline

Goal: 반복 prompt 를 .agents/skills 또는 docs workflow 로 제품화한다.

Initial skills:

/mvp-brief

/codex-preflight

/verify-report

/retrieval-eval

/source-ingest

/release-prep

/compliance-review

Definition of Done:

skills have SKILL.md

commands are tested on one real MVP

AGENTS.md references skill usage

new developer can follow without prior context

8.6 MVP-020: Internal Deployment App

Goal: CLI/local app 에서 내부 사용 가능한 RAG endpoint 또는 UI 로 확장한다.

Tasks:

FastAPI service

local auth or access token

env/secrets separation

Docker compose

healthcheck

audit log

backup/rollback runbook

Definition of Done:

docker compose up works

health endpoint works

sample query returns cited answer

no secret committed

internal deployment runbook exists

9. Deployment Playbook

9.1 Deployment 단계 정의

D0 Local Dev

- developer machine only
- no external users
- test data only or internal controlled data

D1 Internal RC

- tagged prerelease
- verified tests/evals
- limited internal users
- known limitations documented

D2 Internal Staging

- deployment environment
- auth/access control
- monitoring/logging
- rollback
- scheduled backups

D3 Internal Production

- stable internal service
- CI/CD protected
- documented incident response
- source/license/compliance controls active

D4 External Beta

- only external-safe data
- legal/privacy/security review
- user terms and approved claims

현재 v1.0.0-rc1 은 D1 후보로 보는 것이 안전하다.

9.2 Internal RC Checklist

- ☐ main clean
- ☐ full pytest pass
- ☐ artifact verification pass
- ☐ retrieval/eval pass or limitations documented

- [] security/help pass
- [] release readiness checklist pass
- [] smoke test pass
- [] docs/releases notes ready
- [] tag created
- [] GitHub prerelease created
- [] rollback instruction written
- [] known limitations documented

9.3 Internal Deploy Checklist

- [] .env.example exists and no real secrets committed
- [] deployment config separated from dev config
- [] Dockerfile/docker-compose verified
- [] healthcheck endpoint verified
- [] logs path configured
- [] source data path configured
- [] backup path configured
- [] access control enabled
- [] admin runbook written
- [] incident rollback tested

9.4 Rollback Protocol

identify last stable tag

```
git tag --sort=-creatordate | head
```

checkout stable release

```
git checkout v1.0.0-rc1
```

or revert a bad merge commit

```
git checkout main
```

```
git revert <bad_commit_sha>
```

```
python -m pytest
```

```
python scripts/verify_artifacts.py
```

```
git push origin main
```

Rollback report:

Incident:

Detected by:

Bad SHA/tag:

Impact:
Rollback target:
Commands run:
Verification:
Remaining risk:
Next fix ticket:

10. Open Source / External Source Absorption

10.1 Rule

외부 소스는 “복붙 재료”가 아니다. 운영 방식, architecture, test pattern, eval method 를 배우는 benchmark 다. 코드 흡수는 license/security 를 통과해야 한다.

10.2 Protocol

1. Scout
2. Record source URL/repo/paper
3. License check
4. Security/dependency check
5. Maintenance/activity check
6. Benchmark locally
7. Adapt minimal pattern
8. Add NOTICE/LICENSE if required
9. Add tests
10. Record in adoption ledger

10.3 Adoption Ledger

date,source,type,license,what_adopted,files_changed,security_review,test_result,decision,owner
2026-06-25,openai/codex docs,process,N/A,agent-loop/context-management
doctrine,docs/playbook.md,passed,N/A,adopt_as_process,COO

11. Paper / Research Documentation Layer

나중에 논문이나 technical report 를 쓰려면 지금부터 development log 를 논문화 가능한 형식으로 쌓아야 한다.

11.1 기록해야 할 것

문제 정의: 왜 Asperitas agent 가 필요한가?

데이터 구조: source priority, metadata schema, confidentiality design

시스템 구조: ingestion, chunking, embeddings, retrieval, agent harness

평가 방법: golden questions, retrieval metrics, agent traces, human review

결과: before/after performance, failure categories, latency/cost

운영 교훈: prompt vs skill vs workflow vs agent architecture

안전성: compliance gate, human approval, restricted data handling

11.2 논문용 Figure 후보

Figure 1. Asperitas Biological Intelligence Factory architecture

Figure 2. Source registry -> RAG -> agent -> decision log pipeline

Figure 3. MVP development loop with eval gates

Figure 4. Compliance/human approval gate architecture

Figure 5. Retrieval performance before/after by MVP

Figure 6. Agent Factory loop: ChatGPT + Codex + GitHub + Skills

11.3 논문용 Method Skeleton

We developed an internal source-grounded AI agent system using an iterative MVP-based software engineering workflow. Source documents were classified by priority, confidentiality, license status, and verification status. A metadata validation layer preceded ingestion. Documents were extracted, semantically chunked, embedded, and retrieved with source-priority and disclosure-aware filtering. Each MVP was evaluated with unit tests, artifact verification, retrieval metrics, and human review. Changes were managed through GitHub branches, pull requests, required checks, and release tags. High-risk biological, legal, regulatory, investor, and external communication outputs were routed through human approval gates.

12. Standard Templates

12.1 Agent Brief Template

Agent/MVP Name:

Owner:

Date:

Business Objective:

User:
Job-to-be-done:
Current State:
Desired State:
Data Sources:
Source Priority:
Confidentiality Level:
Non-goals:
Risks:
Compliance Gate:
Metrics:
Tests:
Eval Dataset:
Definition of Done:
Next MVP:

12.2 MVP Completion Report

MVP Completion Report

1. Objective:
2. Baseline:
3. Target:
4. Files changed:
5. Behavior changed:
6. Tests run:
7. Eval metrics before/after:
8. Git status:
9. Risks / follow-ups:
10. Decision: ship / iterate / rollback
11. Recommended next MVP:

12.3 PR Body Template

Objective

What changed

Files changed

Tests run

- [] python -m pytest

- [] python scripts/verify_artifacts.py
- [] retrieval eval, if applicable
- [] security/dependency check, if applicable

Eval before/after

Risk / compliance

Rollback plan

Reviewer checklist

- [] Scope is small
- [] Tests pass
- [] No secret/PII leak
- [] No unsupported production claims
- [] Docs updated if behavior changed

12.4 Decision Log Template

date:

decision_id:

context:

sources_used:

evidence_label:

assumptions:

options_considered:

chosen_path:

why_not_alternatives:

risks:

compliance_check:

tests_run:

eval_result:

next_action:

deadline:

owner:

outcome_after_review:

12.5 Release Notes Template

Release vX.Y.Z-rcN

Status

Internal RC / Staging / Production

SHA

Summary

New Features

Fixes

Verification

- pytest:
- artifact verify:
- retrieval eval:
- smoke:
- security:

Known Limitations

Upgrade Notes

Rollback

Human Approval

13. Do Not Build Yet

아래는 아직 만들지 않는다.

autonomous wet-lab execution

live genetic engineering execution agent

external-facing regulatory/legal conclusion generator

investor claim auto-sender

unsupervised patent filing/FTO conclusion system

public chatbot over confidential internal data

web-scale crawler without license/terms review

foundation model claim before proprietary validated data and DBTL loops

14. 7-Day New Developer Onboarding

Day 1 - Mission / Constitution

Read:

README.md

AGENTS.md

P0_ACTIVE_PROMPT summary

this playbook

Task:

Explain truth boundary in your own words.

Identify production vs scaffold status.

Day 2 - Repo / GitHub

Task:

Clone repo.

Run tests.

Inspect branch protection and PR template.

Create a dummy docs branch and PR.

Day 3 - Source Registry

Task:

Add one sample source metadata row.

Run validator.

Explain confidentiality/license decision.

Day 4 - Ingestion / Chunking

Task:

Extract one approved document.

Inspect markdown and chunks.

Verify provenance fields.

Day 5 - RAG / Eval

Task:

Run golden question eval.

Interpret failures.

Propose one retrieval improvement.

Day 6 - Agent / Guardrail

Task:

Run one agent workflow.

Inspect trace/log.

Trigger compliance gate with test query.

Day 7 - MVP PR

Task:

Complete a small docs/test improvement PR.

Include tests and decision log.

Present completion report.

15. Final Operating Doctrine

Asperitas 의 AI Agent 개발은 앞으로 다음 문장으로 고정한다.

Build operating loops, not prompts. Build validated data, not narratives. Build source-grounded decisions, not chatbot answers. Build compliance as infrastructure, not afterthought. Build every MVP as a measurable experiment.

최종 기준:

Scalability: 실험 -> 제품 -> 플랫폼 -> 인프라로 확장되는가?

Moat: access, data, IP, workflow, validation, regulation, partner network 가 누적되는가?

Biosafety/Compliance: CITES, Nagoya, LMO, biosafety, biosecurity, legal, ethics, privacy, auditability 를 보호하는가?

Learning Velocity: 다음 실험과 다음 의사결정을 더 빠르게 만드는가?

Capital Efficiency: 적은 자원으로 불확실성을 줄이는가?

이 기준을 통과하지 못하는 기능은 만들지 않는다. 통과하는 기능은 작게 만들고, 검증하고, 기록하고, 배포한다.

16. 보완사항 - v3.1 Upgrade Backlog

이 장은 v3.0-RC1 이후 반드시 보완해야 할 항목이다. 우선순위는 “배포 안정성 -> 품질 측정 -> 보안/컴플라이언스 -> 자동화 -> AI-native 조직화” 순서다.

16.1 Product / Deployment 보완

현재 v1.0.0-rc1 은 내부 RC 후보로 보는 것이 안전하다. production 이라고 부르려면 아래가 추가로 필요하다.

Production Readiness Gap

- auth/access control: 누가 어떤 source 에 접근 가능한지 권한 분리
- secrets management: .env, API key, GitHub token, DB password 가 repo 에 없다는 보장
- monitoring: latency, error rate, retrieval failure, token cost, guardrail trigger count
- backup/restore: source registry, processed KB, vector DB, logs 복구 절차
- rollback: bad release, bad chunking, bad embedding, bad source ingestion 복구
- incident response: hallucination, data leak, wrong citation, compliance miss 발생 시 대응
- data retention: raw/source/chunk/log 보관 기간과 삭제 정책
- user feedback loop: 답변별 thumbs up/down, correction, missing source request 수집

Definition of Done:

D2 staging 이상에서는 healthcheck, auth, monitoring, backup, rollback, incident runbook 이 없으면 배포 금지.

16.2 RAG 품질 보완

현재 핵심 병목은 “답변이 되는가”가 아니라 “근거를 정확히 찾고, 정확히 인용하고, 모르면 멈추는가”다.

필수 보완:

- golden question set 100 개 이상
- source priority 별 retrieval eval
- citation exactness checker
- unsupported claim detector
- confidentiality leak test
- prompt injection test
- chunking version history
- embedding model/version history
- retrieval configuration registry

권장 파일:

05_EVALS/golden_sets/retrieval_gold_v1.yaml
05_EVALS/reports/retrieval_eval_YYYYMMDD.md
03_PROCESSED_KB/chunking/chunking_config_v1.yaml
03_PROCESSED_KB/embeddings/embedding_manifest.csv

16.3 Agent 품질 보완

Agent 는 기능이 많아질수록 더 똑똑해지는 것이 아니라, 실패 경로가 늘어난다. 그래서 tool, schema, guardrail, trace 가 먼저 강화되어야 한다.

필수 보완:

- tool schema strict validation
- pydantic input/output validation
- router decision log
- agent trace with run_id
- human approval queue
- retry/stop/rollback rule
- unsafe tool-call refusal tests
- external-output draft watermark

Agent run 표준 로그:

```
{  
  "run_id": "20260625-0001",  
  "user_task": "...",  
  "router_decision": "literature_agent",  
  "sources_retrieved": ["ASP-AOS-010", "PUBMED-..."],  
  "tools_called": ["retrieval.search", "citation.check"],  
  "guardrails_triggered": ["biosafety_review"],
```

```
"output_mode": "draft_requires_human_approval",  
"confidence": "medium",  
"human_decision": "pending"  
}
```

16.4 GitHub / Engineering 보완

GitHub 운영은 거의 맞는 방향이다. 다만 RC 이후부터는 아래를 필수화한다.

- branch protection mandatory
- required status checks mandatory
- CODEOWNERS for critical modules
- PR template mandatory
- release notes mandatory
- dependency review mandatory
- SBOM export per release
- secret scanning enabled
- Dependabot or equivalent dependency update workflow
- conventional commit convention

Recommended required checks:

```
pytest  
artifact_verify  
metadata_validate  
retrieval_eval  
compliance_eval  
lint_or_typecheck  
secret_scan  
dependency_review
```

16.5 Documentation Debt 보완

문서가 많아지면 오히려 성능이 떨어질 수 있다. 같은 내용의 AOS, Playbook, README, AGENTS.md 가 서로 충돌하면 agent 가 헛갈린다.

해결 원칙:

- README.md = repo 사용법 / 빠른 시작 / 명령어
- AGENTS.md = AI coding agent 행동 규칙
- PLAYBOOK.md = 사람/신규개발자 운영 매뉴얼
- AOS/PRIME = 전략/헌법/판단 원칙
- docs/mvp/ = 각 MVP 별 실행 기록
- docs/releases/ = 릴리스 기록

중복 제거 규칙:

같은 규칙이 3 곳 이상 반복되면 한 곳을 canonical source 로 지정하고 나머지는 링크만 둔다.

17. 주의사항 - Do Not Break These Rules

17.1 가장 큰 위험: 과장

절대 다음을 말하지 않는다.

- DB 가 완전히 학습됐다.
- foundation model for biology 가 완성됐다.
- production RAG/KG/eval 이 운영 중이다.
- wet-lab validation 이 끝났다.
- 법률/규제 검토가 완료됐다.
- investor-ready claim 이다.

정확한 표현:

- source registry scaffold exists.
- internal RC candidate exists.
- production deployment requires verification.
- claim requires source/eval/human approval.

17.2 생물학/실험 자동화 주의

Asperitas 의 장기 방향은 AI-bio platform 이지만, 지금 만들 시스템은 먼저 decision-support RAG 다. 아래는 human approval 없이 자동화하지 않는다.

- 유전자 조작 프로토콜 실행
- 균주/샘플 이동, 수입, 수출 판단
- CITES/Nagoya/LMO 관련 결론
- 병원성/독성/생물보안 관련 실험 설계
- wet-lab 로봇 실행 명령
- 외부 제출용 규제 문서

17.3 외부자료 수집 주의

좋은 자료를 많이 모으는 것은 맞지만, 아래 규칙을 지킨다.

- 저작권 PDF 무단 크롤링 금지
- 공식 API/공식 페이지 우선
- metadata/abstract/DOI/PMID/PMCID/license 먼저 수집

- 원문 ingestion 은 license/use permission 확인 후
- internal/private file 은 public tool 에 업로드 금지
- 외부 링크는 source_map 에 먼저 등록

17.4 AI agent 과신 금지

Agent 는 execution compressor 이지 책임 주체가 아니다. 특히 다음은 사람의 승인 없이는 금지한다.

- main merge
- release publish
- customer/investor email send
- regulatory/legal conclusion
- confidential document sharing
- irreversible database migration
- production credential change

17.5 복잡한 프레임워크 과사용 금지

초기에는 LangGraph, multi-agent orchestration, KG, autonomous lab 같은 고급 구조를 무조건 넣으면 안 된다. 순서는 다음과이다.

1. single reliable workflow
 2. strict schema
 3. eval and trace
 4. human approval
 5. only then multi-agent/tool automation
-

18. 피드백 - 현재 개발 방식에 대한 냉정한 평가

18.1 잘하고 있는 점

- 처음부터 source-grounded 방향을 잡은 것은 맞다.
- prompt constitution 과 AGENTS.md 를 만든 것은 Codex/Cursor/Claude Code 운용에 유리하다.
- MVP 단위로 쪼개고 pytest/artifact/smoke/release readiness 를 본 것은 좋은 엔지니어링 습관이다.
- v1.0.0-rc1 에서 tag/release command 까지 명확히 간 것은 내부 배포 체계로 전환되는 신호다.
- source map 과 production DB 를 구분하는 truth boundary 를 넣은 것은 매우 중요하다.

18.2 부족한 점

- MVP 별 성능지표가 아직 충분히 고정되지 않았다.
- eval dataset 이 제품 수준으로 축적되지 않았다.
- RAG 성능 개선이 기능 개발과 별도 흐름처럼 느껴진다.

- 자동화는 아직 CI/CD 중심이고, 운영 업무 자동화까지 확장되지 않았다.
- 문서가 빠르게 늘어났기 때문에 canonical source 정리가 필요하다.
- AI-native company 운영법은 아직 문서화 단계이며 실제 업무 시스템에 더 연결해야 한다.

18.3 다음 단계의 핵심 피드백

결론: 이제부터는 “더 많은 기능”보다 “측정 가능한 성능향상 + 운영 자동화 + 조직 루프”가 우선이다.

개발 우선순위:

1. Retrieval eval pack
2. Compliance gatekeeper
3. Trace/decision log
4. Internal deployment app
5. Workflow automation dashboard
6. Daily AI research/market/regulation brief
7. Biofoundry/LIMS-ready data model

Devil's Advocate:

Asperitas 가 지금 실패한다면 기술력이 없어서가 아니라, 문서/프롬프트/아이디어는 많지만 실제 반복 가능한 eval, data pipeline, deployment, user feedback loop 가 부족해서 실패할 가능성이 크다.

19. AI-Native Company 가 되는 방법

19.1 정의

AI-native company 는 “AI 를 쓰는 회사”가 아니다. 회사의 모든 반복 업무가 데이터화되고, 표준화되고, 평가되고, 자동화되며, 사람이 execution worker 가 아니라 system designer/reviewer/operator 가 되는 회사다.

Asperitas 기준 정의:

AI-native Asperitas = 모든 업무가 source -> workflow -> agent -> eval -> decision log -> automation -> human approval loop 로 연결되는 회사

19.2 AI-Native Operating System 6-Layer Architecture

Layer 1. Work Map

- 회사의 반복 업무를 전부 목록화한다.
- 예: 논문 모니터링, 규제 확인, 소스 등록, IR 업데이트, 실험계획 검토, 회의록 정리.

Layer 2. Knowledge Layer

- 모든 지식은 source registry, metadata, RAG, KG-ready schema 로 관리한다.
- 문서가 아니라 governed knowledge asset 으로 본다.

Layer 3. Workflow Layer

- 업무를 SOP, checklist, template, command, queue 로 만든다.
- 사람이 매번 설명하지 않아도 실행 가능해야 한다.

Layer 4. Agent Layer

- 각 workflow 에 맞는 agent 를 붙인다.
- 단일 거대 agent 보다 작은 역할 agent 가 안전하다.

Layer 5. Eval / Observability Layer

- AI output 을 믿지 말고 측정한다.
- trace, score, failure mode, correction 을 남긴다.

Layer 6. Governance Layer

- 권한, 보안, 컴플라이언스, human approval, audit log 를 기본 인프라로 둔다.

19.3 Asperitas AI-Native 조직 설계

COO / AI Manager

- AI operating system owner
- priority, risk, approval, KPI 결정

AI Researcher

- literature/source acquisition
- benchmark/eval dataset
- research brief pipeline

AI Engineer / Codex Operator

- repo, RAG, eval, app, CI/CD 구현

Bio Domain Reviewer

- biological mechanism, DBTL, wet-lab feasibility 검토

Compliance Reviewer

- CITES/Nagoya/LMO/biosafety/IP/legal gate 검토

Business Reviewer

- IR, market, partner, commercialization claim 검토

최소 인원일 때는 한 사람이 여러 역할을 겸해도 된다. 단, 역할 분리는 문서와 체크리스트에서 반드시 유지한다.

19.4 AI-Native 업무 원칙

1. 반복되는 일은 SOP 로 만든다.
2. SOP 가 3 번 반복되면 template 으로 만든다.
3. template 이 3 번 반복되면 script 로 만든다.
4. script 가 안정되면 GitHub Actions/cron 으로 자동화한다.
5. 자동화가 판단을 포함하면 agent + eval + human approval 을 붙인다.
6. 모든 결과는 decision log 로 남긴다.

19.5 100-Day AI-Native 전환 계획

Day 1-15: Work Map

- 모든 반복 업무 50 개 이상 목록화
- 시간 소모, 빈도, 리스크, 자동화 가능성 점수화

Day 16-30: Knowledge Layer

- source registry 정리
- metadata schema 확정
- P0/P1/P3/P4 핵심 자료 ingestion/eval

Day 31-45: Workflow Layer

- 논문 브리핑, 규제 브리핑, source ingest, release readiness SOP 제작
- prompt library 와 PR template 정리

Day 46-60: Agent Layer

- Literature Agent, Compliance Gatekeeper, RAG Answer Agent, Release Assistant MVP

Day 61-75: Eval Layer

- golden questions 100 개
- compliance eval 50 개
- retrieval regression report 자동 생성

Day 76-90: Automation Layer

- GitHub Actions 자동 검증
- scheduled research brief
- source manifest generator
- release checklist generator

Day 91-100: Governance Layer

- access control
 - incident runbook
 - human approval dashboard
 - internal training/onboarding
-

20. AI Master 가 되기 위한 공부 로드맵

20.1 목표 정의

AI Master 는 프롬프트를 잘 쓰는 사람이 아니다. AI 를 이용해 회사의 지식, 코드, 실험, 의사결정, 자동화, 제품을 compounding system 으로 만드는 사람이다.

Asperitas AI Master 의 역량 공식:

AI Mastery = AI literacy + software engineering + data/RAG + evals + automation + domain biology + compliance + product judgment

20.2 Level 0 - 기본 도구

필수:

- Python: 파일 처리, API 호출, 데이터 처리, 테스트
- Git/GitHub: branch, commit, PR, issue, release, Actions
- Shell/PowerShell: 파일 이동, 압축, 실행, 로그 확인
- Markdown/YAML/JSON: agent prompt, config, metadata 작성
- SQL: source registry, logs, eval result 조회
- Docker: local app, reproducible environment

실습 과제:

- source metadata CSV validator 만들기
- GitHub Actions 로 pytest 자동 실행하기
- PDF/DOCX 파일을 markdown 으로 추출해 source_id 붙이기

20.3 Level 1 - LLM / Prompt / Agent 기본

공부할 개념:

- token / context window / attention 의 실무적 의미
- system/developer/user instruction 분리
- structured output / JSON schema
- tool calling
- RAG

- embeddings
- reranking
- prompt injection
- hallucination control
- agent loop
- workflow vs autonomous agent 차이

실습 과제:

- 하나의 업무를 prompt -> checklist -> script -> agent 로 단계별 변환
- 같은 질문을 RAG 없음/RAG 있음으로 비교
- citation 없는 답변을 자동 fail 시키는 eval 만들기

20.4 Level 2 - Evals / Reliability

공부할 개념:

- golden dataset
- unit eval vs integration eval
- LLM-as-judge 의 한계
- regression testing
- precision/recall
- false positive/false negative
- adversarial cases
- trace review

실습 과제:

- Asperitas RAG golden question 100 개 만들기
- retrieval top-k recall 측정
- compliance false negative case 50 개 만들기
- before/after eval report 자동 생성

20.5 Level 3 - Software / Production

공부할 개념:

- FastAPI
- PostgreSQL / pgvector
- background jobs
- logging / monitoring
- auth / access control
- CI/CD
- secrets management

- dependency security
- SBOM
- backup/rollback

실습 과제:

- FastAPI RAG endpoint 만들기
- Docker compose 로 local deployment
- GitHub Actions 에 metadata_validate + retrieval_eval 추가
- release tag 와 notes 자동 생성

20.6 Level 4 - Automation / Operations

공부할 개념:

- GitHub Actions
- cron/scheduler
- Slack/Gmail/Calendar automation
- RSS/PubMed/Crossref/SEC/Patents API 기반 source manifest
- webhook
- task queue
- human-in-the-loop approval
- dashboard

실습 과제:

- 매일 아침 literature/regulatory/company signal brief 생성
- 새 source 가 추가되면 metadata validation 자동 실행
- PR 이 열리면 Codex/Claude review 요청 checklist 생성
- release 전에 readiness report 자동 생성

20.7 Level 5 - Bio-AI / Synthetic Biology Domain

공부할 개념:

- central dogma / gene / protein / pathway
- DBTL cycle
- omics data: genome, transcriptome, proteome, metabolome
- protein design / enzyme engineering
- strain engineering
- biofoundry automation
- LIMS/ELN
- sample/protocol/version tracking

- biosafety/biosecurity
- CITES/Nagoya/LMO/IP basics

실습 과제:

- species -> gene -> protein -> pathway -> product claim card 설계
- DBTL experiment registry schema 설계
- wet-lab protocol 을 바로 실행하지 않고 review-ready protocol card 로 변환
- compliance gate eval set 제작

20.8 Level 6 - Executive AI Strategy

공부할 개념:

- AI-native company design
- platform strategy
- data flywheel
- moat architecture
- product-market fit
- IR claim governance
- ROI/KPI measurement
- build/buy/partner decision

실습 과제:

- Asperitas AI roadmap 을 Phase 0-3 로 재작성
- 각 agent 의 business KPI 정의
- 자동화 전/후 시간 절감과 리스크 감소 측정
- investor-safe AI capability claims registry 작성

20.9 주간 학습 루틴

월: AI engineering 공식 문서 1 개 + 요약

화: 논문/벤치마크 1 개 + Asperitas 적용점

수: Codex/Claude/GitHub 로 작은 자동화 구현

목: eval/golden set 개선

금: decision log / playbook 업데이트

토: bio domain 공부

일: 전체 시스템 리뷰 및 다음 주 MVP 설계

21. 자동화 방법 - Asperitas Automation Ladder

21.1 자동화 단계

Level 0. Manual

- 사람이 매번 직접 수행

Level 1. Checklist

- 실수 방지를 위해 체크리스트화

Level 2. Template

- 반복 문서/프롬프트/보고서 양식화

Level 3. Script

- Python/PowerShell/bash 로 반복 작업 실행

Level 4. CI/Scheduled Job

- GitHub Actions, cron 등으로 자동 실행

Level 5. Event-Triggered Workflow

- 파일 추가, PR 생성, 이메일 수신, RSS 업데이트 등 이벤트 기반 실행

Level 6. Agent-Assisted Workflow

- AI 가 분석/요약/분류/초안을 수행하고 사람이 승인

Level 7. Human-Approved Autonomy

- 안전한 범위 내에서 AI 가 실행까지 수행하되, high-risk 는 approval gate

21.2 우선 자동화할 업무 Top 15

1. pytest + artifact verify + metadata validate GitHub Actions
2. PR 생성 시 자동 checklist/decision log draft
3. release readiness report 자동 생성
4. source metadata CSV validation
5. new raw source checksum 생성
6. PDF/DOCX/PPTX extraction to markdown
7. chunk generation + chunk manifest 생성
8. retrieval golden eval 자동 실행
9. compliance trigger eval 자동 실행
10. daily literature/RSS/PubMed brief
11. daily regulation/government update brief

12. competitor/company monitoring brief
13. weekly KPI/cost/latency report
14. backup and restore test reminder
15. onboarding task generator for new developer

21.3 GitHub Actions 예시

name: **verify**

on:

pull_request:

branches: [**main**]

workflow_dispatch:

jobs:

test:

runs-on: **ubuntu-latest**

steps:

- uses: **actions/checkout@v4**

- uses: **actions/setup-python@v5**

with:

python-version: **'3.11'**

- run: **python -m pip install -r requirements.txt**

- run: **python -m pytest**

- run: **python scripts/validate_metadata.py**

- run: **python scripts/verify_artifacts.py**

- run: **python scripts/run_retrieval_eval.py**

21.4 Daily Research Brief 자동화 설계

Input:

- PubMed / Europe PMC / Crossref metadata
- government/regulatory official RSS or pages
- company official news / SEC filings
- selected industry sources

Pipeline:

1. collect metadata only
2. deduplicate by DOI/title/URL
3. classify by domain: synbio, DBTL, biofoundry, protein design, regulation, competitor
4. score relevance to Asperitas
5. generate Korean executive brief

6. attach source links and license status
7. route high-risk/regulatory items to human review

Output:

- docs/briefs/YYYYMMDD_daily_bio_ai_brief.md
- source_map_delta.csv
- action_items.yaml

21.5 Automation Safety Rules

- 자동화는 draft 를 만들 수 있지만 high-risk final action 은 사람이 승인한다.
 - source ingestion 자동화는 license/confidentiality gate 를 건너뛰면 안 된다.
 - AI 가 생성한 data correction 은 원본을 덮어쓰지 않고 patch proposal 로 남긴다.
 - 외부 발송은 기본값 draft-only.
 - production DB migration 은 자동 실행 금지.
-

22. 다음 개발 방향 - Phase 0 to Phase 3

22.1 Phase 0 - Internal Source-Grounded RAG Agent

목표:

내부 문서와 외부 검증 소스를 정확히 읽고, 출처 기반으로 답변하며, 위험하면 human approval 로 올리는 내부 의사결정 시스템.

핵심 산출물:

source registry
metadata validator
extraction/chunking pipeline
vector retrieval
citation checker
golden evals
compliance gatekeeper
trace/decision log
internal CLI or FastAPI app

22.2 Phase 1 - AI Operating System for Company Workflows

목표:

COO/전략/연구/규제/IR/문헌조사 업무를 매일 자동 보조하는 AI-native operating layer.

핵심 산출물:

daily brief agents
workflow automations
PR/release automation
source update pipeline
meeting/decision log automation
internal dashboard

22.3 Phase 2 - Biofoundry / DBTL Data Infrastructure

목표:

실험을 자동 실행하기 전에 sample, protocol, assay, result, hypothesis, decision 을 version-controlled data asset 으로 만든다.

핵심 산출물:

experiment registry
protocol card schema
sample/entity schema
assay result schema
DBTL loop tracker
LIMS/ELN-ready adapter
human-reviewed protocol design agent

22.4 Phase 3 - Biological Intelligence Factory

목표:

biodiversity access -> biological data -> model -> experiment -> validation -> IP -> product -> compliance trust -> market adoption 을 반복 가능한 플랫폼으로 만든다.

핵심 산출물:

KG + RAG + DBTL loop integration
proprietary biological dataset
model-assisted design
closed-loop experiment planning, still human-approved
IP/compliance claim registry
commercialization intelligence dashboard

23. v3.1 Copy-Paste Master Prompt for Next Development

Role:

You are the AI engineering lead for the private Asperitas AI RAG Agent / AI Agent Factory.

Context:

We have an internal RC-level source-grounded RAG/agent repository. The operating doctrine is AOS/PRIME + Agent Factory Playbook v3.1-RC1. The target is not a generic chatbot; it is a source-grounded, auditable, compliance-aware Biological Intelligence Factory foundation.

Task:

For the next MVP, do not only add features. Build the MVP with its Performance Pack, evals, GitHub verification, documentation update, and automation opportunity.

Hard Rules:

- Inspect before editing.
- Make the smallest safe change.
- Preserve source provenance, confidentiality, license status, and verification status.
- Separate scaffold from production.
- Do not claim legal/regulatory/wet-lab/foundation-model completion unless verified.
- Trigger human approval for biosafety, biosecurity, CITES, Nagoya, LMO, legal, investor, IP, external communication, or wet-lab-sensitive outputs.

Required Output:

1. Preflight summary
2. Proposed file/module changes
3. MVP Performance Pack
4. Test plan
5. Eval plan
6. Automation opportunity
7. Risk/compliance gate
8. Definition of Done
9. Commands to run
10. Decision: execute / split / verify first / stop

Stop Rule:

If repo state, source status, or risk boundary is unclear, stop and ask before implementation.

24. v3.1 External Reference Anchors

These references are not internal Asperitas facts. They are operating benchmarks for agent engineering, evaluation, GitHub governance, and AI-native company design. Use them as implementation patterns only.

- E1. OpenAI Codex web documentation - GitHub connection, repo work, PR creation.
- E2. OpenAI AGENTS.md guide - repository-specific guidance for Codex.
- E3. OpenAI evals and evaluation best practices - eval-driven AI system reliability.
- E4. OpenAI Practical Guide to Building Agents - use case selection, agent orchestration, guardrails, cost/latency optimization.
- E5. Anthropic Building Effective Agents - simple composable workflows before complex frameworks.
- E6. Anthropic Claude Code Skills/Subagents/Hooks/MCP docs - reusable skills, role-specialized agents, automation hooks.
- E7. GitHub branch protection, dependency review, SBOM documentation - protected release and supply-chain governance.
- E8. GitHub Copilot cloud agent documentation - branch-based implementation planning and PR review workflow.

Reference Anchors

Internal anchors:

ASPERITAS AGENT FACTORY PLAYBOOK v2.0, 2026-06-15.

Asperitas Codex AI Agent Build Guidebook v1.0, 2026-06-10.

P0_ACTIVE_PROMPT Master Constitution Bundle, 2026-06-09.

ASPERITAS PRIME v2.0 Universal Master System Prompt.

User-provided v1.0.0-rc1 verification report, 2026-06-25 conversation context.

External official anchors checked for this playbook:

OpenAI Codex cloud / GitHub setup documentation.

OpenAI Codex GitHub code review documentation.

OpenAI Codex agent loop technical note.

OpenAI prompt engineering and eval best practices documentation.

Anthropic Claude Code overview, Skills, Subagents, and MCP documentation.

GitHub branch protection, required status checks, dependency review, and SBOM documentation.

25. Perfect AI Engineer Skill Stack

25.0 Executive Bottom Line

완벽한 AI engineer 는 단순히 Python 을 잘 쓰는 사람이 아니다. Asperitas 기준의 AI engineer 는 **제품 요구사항을 eval 가능한 시스템으로 바꾸고, repo 에서 안전하게 구현하고, 성능을 측정하고, 실패를 재현하고, GitHub 에 의사결정을 남기며, 자동화 가능한 절차를 Skills/Workflow/CI 로 고정하는 사람**이다.

핵심 공식:

AI Engineer = Product Spec + Data Architecture + Model/RAG/Agent Design + Eval + Software Engineering + Security + Deployment + Ops Automation + Domain Judgment

일반 개발자와 AI engineer 의 차이:

일반 개발자: 요구사항 -> 코드 -> 테스트 -> 배포

AI engineer: 문제정의 -> 데이터/도구/모델 선택 -> baseline -> eval 설계 -> 구현 -> trace -> failure analysis -> prompt/tool/retrieval/model 개선 -> regression test -> deployment -> monitoring -> feedback loop

Asperitas 에서는 여기에 Bio-AI 와 compliance 가 추가된다.

Asperitas AI Engineer = AI Engineer + Source Governance + Bio Domain Model + DBTL Logic + Biosafety/Compliance Gate + IP/Regulatory Awareness

25.1 Perfect AI Engineer 의 12 대 기술

1) Problem Framing

기능 요청을 바로 코드로 바꾸지 않는다. 먼저 문제를 다음 형식으로 잠근다.

User / Buyer / Operator

Job-to-be-done

Current pain

Target behavior

Non-goals

Success metric

Failure metric

Risk boundary

Definition of Done

잘못된 방식:

RAG 답변을 더 좋게 해줘.

올바른 방식:

Internal strategy/R&D query 에서 citation precision 을 높인다. Baseline 20 개 golden query 에서 source-support 오류를 측정하고, retrieval top-k/rerank/chunking 변경 전후를 비교한다. latency +20% 이상이면 실패다.

2) Data and Source Engineering

AI 성능의 절반은 모델이 아니라 데이터 구조에서 결정된다.

필수 역량:

metadata schema

source registry

license/confidentiality classification

redaction pipeline

chunking strategy

deduplication

claim extraction

embedding/reranking

citation verifier

knowledge graph staging

Asperitas rule:

자료를 넣었다 = 학습됨 아님.

폴더에 있다 = production DB 아님.

요약했다 = 근거화 아님.

RAG 에 넣었다 = 법적 사용 승인 아님.

3) Retrieval Engineering

RAG 성능향상은 “더 많은 파일 업로드”가 아니라 retrieval chain 전체 최적화다.

개선 레버:

query rewrite

hybrid search

metadata filter

chunk size / overlap

parent-child chunking

reranker

citation span extraction

source-priority weighting

answer grounding checker
abstention rule

측정 지표:

retrieval recall@k
citation precision
faithfulness
answer completeness
abstention correctness
latency
cost per answer

4) Prompt and Instruction Engineering

좋은 prompt 는 긴 prompt 가 아니다. 반복 가능하고 검증 가능한 prompt 다.

필수 구조:

Role
Context
Task
Evidence / RAG
Constraints
Output Schema
Verification
Stop Rules

AGENTS.md 에는 영구 원칙만 넣고, 긴 작업 지시는 task prompt 로 분리한다. AGENTS.md 가 커질수록 성능이 좋아지는 것이 아니라, 반복 오류를 줄이는 정확한 규칙이 추가될 때만 좋아진다.

5) Agent Architecture

Agent 는 “모델 하나”가 아니라 다음의 조합이다.

LLM
system instruction
tools
memory/context
retrieval
planner/router
guardrails
evals
logs
human approval gate

Asperitas 기본 agent pattern:

User Request

-> Intent Router

-> Source Retrieval

-> Risk Classifier

-> Specialist Agent

-> Evidence Checker

-> Compliance Gate

-> Final Answer / Escalation

-> Decision Log

6) Eval Engineering

성능향상은 eval 없이는 불가능하다.

Eval 의 최소 단위:

Golden Query

Expected Source

Expected Behavior

Allowed Answer

Forbidden Answer

Scoring Rule

Failure Category

AI agent eval 은 세 층으로 본다.

Output eval: 답이 맞는가?

Trace eval: 올바른 도구/검색/승인 흐름을 탔는가?

Outcome eval: 실제 업무 시간이 줄고 리스크가 줄었는가?

7) Software Engineering Fundamentals

AI 로 코딩해도 기본기는 절대 생략하지 않는다.

필수:

Python

TypeScript basics

Git / GitHub

CLI

Docker

FastAPI

pytest

ruff / mypy
SQL
YAML / JSON schema
logging
error handling
CI/CD

AI coding agent 를 쓰면 더욱 중요하다. AI 가 만든 코드를 사람이 이해하지 못하면, 속도가 아니라 위험이 늘어난다.

8) Observability and Traceability

AI system 은 작동 여부보다 “왜 그렇게 작동했는지”가 중요하다.

필수 로그:

request_id
user/task type
retrieved sources
model/tool calls
guardrail decisions
eval scores
latency/cost
human approval status
final answer hash
failure tags

9) Security and Supply Chain

AI engineer 는 secret leakage, prompt injection, dependency risk, tool permission 문제를 기본으로 막아야 한다.

필수 체크:

no API keys in repo
.env ignored
secret scanning
least privilege tokens
dependency review
SBOM
license review
MCP/tool allowlist
PII redaction
external upload ban for restricted files

10) Deployment and Release Engineering

배포는 “작동하는 코드”가 아니라 “복구 가능한 시스템”을 내보내는 것이다.

기본 루프:

- branch
- small diff
- local tests
- artifact verification
- eval run
- security check
- PR
- review
- merge
- tag
- release notes
- rollback plan
- monitoring

11) Automation Engineering

반복 작업을 자동화하되, 판단 작업은 바로 자동화하지 않는다. 먼저 checklist 로 표준화하고, 그 다음 script/CI/workflow/agent 로 올린다.

Manual -> Checklist -> Script -> CI -> Scheduled Job -> Event Trigger -> Human-approved Agent -> Limited Autonomy

12) Domain Judgment

Asperitas 에서는 AI engineer 가 bio/compliance 문맥을 모르면 위험하다.

필수 도메인:

- synthetic biology basics
- DBTL cycle
- biofoundry automation
- omics data
- protein/gene/pathway fundamentals
- CITES/Nagoya/LMO/biosafety concepts
- IP/source provenance
- experimental validation boundary

26. MVP + Performance Integration System

26.1 핵심 원칙

다음 개발부터는 MVP 와 성능향상을 분리하지 않는다. 각 MVP 는 반드시 다음 2 개를 같이 낸다.

MVP Feature Pack = 새 기능 / 버그 수정 / workflow 개선

MVP Performance Pack = baseline / metric / eval / optimization / regression guard

즉, “MVP16 개발”은 기능 추가만이 아니다.

MVP16 = Feature + Eval + Performance + Automation + Documentation + GitHub Evidence

26.2 MVP 마다 반드시 붙는 Performance Pack

1. Baseline

- 현재 성능 수치
- 현재 실패 사례
- 현재 latency/cost
- 현재 eval pass rate

2. Hypothesis

- 무엇을 바꾸면 왜 좋아지는가?
- retrieval, prompt, model, schema, tool, UX 중 어느 레버인가?

3. Metric

- 정확도/완성도/출처정확도/속도/비용/안전성 중 무엇을 측정하는가?

4. Implementation

- 최소 변경 원칙
- affected files
- rollback 가능한 구조

5. Verification

- pytest
- golden eval
- artifact verification
- manual smoke
- git diff check

6. Regression Guard

- 이전에 잘 되던 5~20 개 질문이 망가지지 않는지 확인

7. Decision Log

- 개선됨 / 불확실 / 악화 / 보류
- 다음 action

26.3 MVP Definition of Done v3.2

이제 MVP 완료는 다음 조건을 모두 통과해야 한다.

- [] Feature works locally.
- [] Existing tests pass.
- [] New/changed behavior has test or eval.
- [] Performance baseline and after metric recorded.
- [] Source/citation behavior verified if RAG-related.
- [] Security/compliance risk reviewed.
- [] Git diff reviewed.
- [] Decision log updated.
- [] README/AGENTS/Skill/Runbook updated if repeated workflow changed.
- [] PR or commit message states why, what changed, how verified.

26.4 통합 예시

나쁜 MVP:

MVP16: citation UI 개선.

좋은 MVP:

MVP16: citation UI + citation accuracy performance pack.

Goal: answer card 에 source priority/confidence/verification status 표시.

Baseline: 20 golden questions 중 citation mismatch 4 건.

Change: answer formatter + citation checker + UI schema.

Metric: mismatch $\leq 1/20$, latency +10% 이내.

Test: formatter unit test, golden eval, dry-run chat.

DoD: pytest pass, eval report saved, decision log updated, PR ready.

27. ChatGPT / Codex / Claude 분리 운영 프로토콜

27.1 왜 분리해야 하는가

Codex 채팅은 코드 실행과 repo 변경에 집중해야 한다. ChatGPT 는 사령탑이다. Claude 는 long-context review, alternative design, refactor reasoning 에 강하게 쓴다.

운영 원칙:

ChatGPT = command center / strategy / prompt architect / evaluator

Codex = implementation worker / repo editor / test runner / PR maker

Claude = independent reviewer / long-context debugger / skill-subagent designer

GitHub = memory / evidence / release governance

27.2 Codex 채팅은 분리해도 과정은 ChatGPT 에서 명시한다

각 Codex 세션은 분리해도 된다. 그러나 개발 과정과 의사결정 기준은 ChatGPT 사령탑 문서에서 통제한다.

표준 흐름:

1. ChatGPT 에서 MVP Brief 작성
2. ChatGPT 에서 Codex Prompt 생성
3. Codex 새 채팅/새 task 에서 repo 작업
4. Codex 가 report 반환
5. ChatGPT 에서 report 검토/리스크 분석/다음 prompt 생성
6. GitHub PR/commit/release 로 evidence 저장
7. 반복 오류는 AGENTS.md/Skills/Runbook 에 반영

27.3 채팅 분리 기준

새 Codex 채팅을 열어야 하는 경우:

- 다른 branch / 다른 MVP 로 넘어갈 때
- 이전 세션에 로그/에러/탐색 결과가 너무 쌓였을 때
- 구현 작업과 리뷰 작업을 분리할 때
- read-heavy exploration 과 write-heavy implementation 을 분리할 때
- 위험한 migration/security/deployment 작업을 별도 승인 흐름으로 분리할 때

같은 채팅에서 이어가도 되는 경우:

- 같은 PR 의 작은 수정
- test failure 를 고치는 반복
- 방금 만든 diff 에 대한 소규모 리팩터링
- 같은 branch 내 documentation update

27.4 Handoff Packet

Codex/Claude 새 채팅 시작 시 반드시 붙이는 최소 패킷:

Project:

Asperitas AI RAG Agent / AI Agent Factory.

Operating Rule:

Follow AGENTS.md, AOS/PRIME, and Playbook v3.2. Preserve truth boundary: scaffold != production.

Current Task:

<one clear MVP objective>

Repo State:

<branch, SHA if known, latest tests if known>

Files to inspect first:

<list>

Hard Constraints:

- Smallest safe diff.
- Do not delete source/governance files.
- Preserve metadata/source provenance.
- Add tests/evals for changed behavior.
- No external upload of restricted data.

Required Report:

1. Files inspected
2. Files changed
3. Tests run
4. Eval/performance result
5. Risks
6. Next action

27.5 Context Rot 방지

긴 채팅은 성능이 떨어질 수 있다. 대화가 길어질수록 중요한 요구사항이 중간 로그에 묻힌다. 따라서 main 사령탑 문맥에는 결정과 결과만 남긴다.

Raw logs -> Codex thread

Decision summary -> ChatGPT

Permanent rule -> AGENTS.md / Skill / Runbook

Evidence -> GitHub PR / release note / eval report

28. OpenAI / Claude / Microsoft / MIT / Stanford 학습 흡수 지도

28.1 OpenAI 에서 흡수할 것

공식 문서 기준 핵심 학습:

Codex web/cloud

AGENTS.md custom instructions

Codex GitHub code review

Codex skills

Codex MCP

Codex subagents

OpenAI evals

Agent workflow evals with traces/graders/datasets

Asperitas 적용:

- Codex cloud: background implementation and PR creation.
- AGENTS.md: repo-level operating doctrine.
- GitHub review: PR 마다 P0/P1 risk review.
- Skills: repeated workflows such as source ingestion, eval report, release prep.
- MCP: trusted internal tools only, allowlist and permission gates.
- Subagents: read-heavy exploration, test triage, log analysis, not uncontrolled parallel writing.
- Evals: every MVP has golden queries and trace-level checks.

공부 순서:

1. Codex web and GitHub connection
2. AGENTS.md hierarchy
3. Codex code review and PR workflow
4. Evals / graders / traces
5. Skills and plugin structure
6. MCP with security constraints
7. Subagent workflow design

28.2 Anthropic / Claude 에서 흡수할 것

핵심 학습:

Claude Code as agentic coding environment

Skills as packaged expertise

Hooks as deterministic automation

Subagents as isolated specialist contexts

MCP as external tool/data connection layer

Agent evals with code-based, model-based, human graders

Asperitas 적용:

- Skills: 문서 작성, source registry, eval generation, release note, compliance review 를 재사용 skill 로 만든다.
- Hooks: format/test/notification/security scan 처럼 deterministic 한 것은 hook 으로 고정한다.
- Subagents: Literature reviewer / Security reviewer / Eval designer / Refactor reviewer 처럼 분리한다.
- MCP: Google Drive/GitHub/Docs/DB/ELN 연결은 직접 연결 전 permission/risk model 부터 설계한다.

주의:

Claude/agent 가 할 수 있다고 바로 자동화하지 않는다.

코드 실행/파일 접근/외부 도구 접근이 생기면 보안 경계가 열린다.

모든 hooks/skills/MCP 는 allowlist, secret policy, restricted data policy 가 필요하다.

28.3 Microsoft 에서 흡수할 것

핵심 학습:

Microsoft Foundry / Azure AI Foundry

AI learning paths

AI Engineer career path

Responsible AI practices

enterprise operations: RBAC, monitoring, tracing, evaluations, policy, networking

Asperitas 적용:

- AI-native company 가 되려면 앱 하나가 아니라 enterprise AI operations 구조가 필요하다.
- RBAC, monitoring, eval, audit, policy, deployment environment 를 초기부터 설계한다.
- 나중에 enterprise partner, pharma, government R&D 로 갈 때 Azure/GitHub/Foundry 식 governance 언어가 유리하다.

공부 순서:

1. AI fundamentals
2. Azure AI / Foundry basics
3. Generative AI app development
4. Responsible AI
5. AI Engineer path
6. Monitoring/evals/RBAC/networking
7. Enterprise deployment patterns

28.4 MIT 에서 흡수할 것

핵심 학습:

AI 101 / Artificial Intelligence fundamentals

Algorithms and computational thinking

MIT 6.S191 Introduction to Deep Learning

Generative AI and agentic AI professional education

Asperitas 적용:

- 단기: AI 기본 개념, neural networks, Transformers, generative models, Python labs.
- 중기: deep learning 을 RAG/agent/eval 성능 개선에 연결.
- 장기: biology/medicine application 까지 연결해 Bio-AI engineer 로 확장.

학습 포인트:

linear algebra

calculus intuition

probability/statistics

optimization

deep sequence modeling

computer vision

LLM/generative modeling

reinforcement learning basics

biology/medicine applications

28.5 Stanford 에서 흡수할 것

핵심 학습:

CS229 - Machine Learning foundations

CS224N - NLP with Deep Learning and LLMs

CS231N - Deep Learning for Computer Vision

CS25 - Transformers / foundation models orientation

Stanford HAI AI Index - macro trends, governance, economy, science, medicine, education

Asperitas 적용:

- CS229: ML 수학과 모델링 기본기.
- CS224N: LLM/RAG/semantic retrieval/transformer 이해.
- CS231N: bioimage, phenotype, microscopy, organoid image pipeline 대비.
- HAI AI Index: 투자자/전략/정책 브리핑의 객관 데이터 레이어.

28.6 GitHub 에서 흡수할 것

핵심 학습:

branch protection

required status checks

pull request review

dependency review

secret scanning

SBOM

release tags

GitHub Actions

Copilot/Codex/agent workflows

Asperitas 적용:

- main branch 는 사람 승인과 required checks 없이는 merge 금지.
 - dependency change 는 자동 리뷰.
 - release 는 tag + release notes + artifact verification + rollback note 포함.
 - AI-generated PR 도 일반 PR 보다 더 엄격하게 본다.
-

29. AI Master 12-Week Study Roadmap

29.0 목표

12 주 후 목표는 “AI 용어를 아는 사람”이 아니라, 다음을 혼자 설계할 수 있는 수준이다.

문제 정의 -> 데이터 구조 -> RAG/Agent 설계 -> eval 설계 -> Codex 구현 지시 -> 테스트/배포 -> 자동화 -> 리스크 관리

29.1 Week 1 - Computer / Git / CLI 기본기

학습:

terminal basics

file paths

Git branch/commit/merge/PR

GitHub issues/projects/actions overview

Markdown/YAML/JSON

실습:

- repo clone
- branch 생성
- README 수정
- PR 생성
- conflict 해결
- .gitignore/.env 이해

29.2 Week 2 - Python Engineering

학습:

Python syntax
functions/classes
venv/uv/pip
pytest
logging
argparse/typer
pydantic
FastAPI basics

실습:

metadata validator 만들기
source registry CSV validation
unit test 작성

29.3 Week 3 - Data / Search / RAG Foundations

학습:

text extraction
chunking
embeddings
vector search
hybrid search
reranking
metadata filters
citation grounding

실습:

10 개 문서를 markdown 으로 추출
chunks 생성

retrieval test 20 개 작성
citation mismatch 기록

29.4 Week 4 - Prompt / Instruction / AGENTS.md

학습:

Role/Context/Task/Evidence/Constraints/Format/Verification
few-shot
negative constraints
schema-first prompting
AGENTS.md hierarchy
Stop rules

실습:

AGENTS.md 개선 PR 만들기
Codex task prompt 3 종 작성
반복 오류 3 개를 rule 로 전환

29.5 Week 5 - LLM / Transformer / Deep Learning Core

학습:

neural network basics
attention
Transformers
sequence modeling
LLM inference concepts
token/context/latency/cost

실습:

MIT 6.S191 lectures/labs 중 sequence, vision, generative modeling 핵심 수강
Stanford CS224N attention/transformer 강의 수강

29.6 Week 6 - Eval Engineering

학습:

golden sets
code-based grader
model-based grader
human grader
trace eval
regression eval

calibration
failure taxonomy

실습:

20 개 golden question 작성
answer_quality, citation_precision, abstention_correctness 채점표 작성
baseline report 생성

29.7 Week 7 - Agent Architecture

학습:

agent loop
planner/router
tool calling
guardrails
handoffs
memory/context management
subagents
human-in-the-loop

실습:

Research Agent -> Compliance Gate -> Answer Formatter workflow 설계
trace log schema 작성

29.8 Week 8 - Codex / Claude / GitHub Agentic Development

학습:

Codex web/cloud
Codex GitHub code review
AGENTS.md
Claude Skills
Claude Hooks
subagents
MCP basics

실습:

one MVP branch
Codex implementation prompt
Claude review prompt

GitHub PR

@codex review style checklist

29.9 Week 9 - Security / Compliance / Responsible AI

학습:

secret management

PII redaction

prompt injection

tool permission

MCP security

license review

CITES/Nagoya/LMO/biosafety boundary

responsible AI principles

실습:

restricted data policy 작성

MCP allowlist 초안 작성

compliance gate test cases 10 개 작성

29.10 Week 10 - Automation / CI / Deployment

학습:

GitHub Actions

scheduled workflows

artifact verification

release tags

Docker basics

FastAPI deployment basics

monitoring

rollback

실습:

pytest GitHub Action

metadata validation CI

release readiness script

29.11 Week 11 - Bio-AI / Synthetic Biology Integration

학습:

central dogma
genes/proteins/pathways
omics data
DBTL
biofoundry automation
protein design basics
organoid/scalable biomanufacturing concepts

실습:

DBTL protocol card schema 작성
Bio-AI source registry tags 설계
claim card: fact/inference/speculation 분류

29.12 Week 12 - Capstone

최종 과제:

Build one complete MVP with:

- MVP Brief
- source/data changes
- implementation prompt
- code/test/eval
- performance report
- security/compliance review
- PR/release note
- decision log
- automation opportunity

통과 기준:

혼자서 다음 MVP 를 설계하고, Codex 에 시키고, 결과를 검증하고, 성능향상까지 판단할 수 있다.

30. Automation Architecture and Implementation Ladder

30.1 자동화의 핵심 기준

자동화는 “사람이 하기 귀찮은 것”부터 하는 게 아니다. 다음 조건을 만족하는 것부터 한다.

반복 빈도가 높다.

입력/출력 형식이 안정적이다.

성공/실패 판정이 명확하다.

실패해도 복구 가능하다.

보안/컴플라이언스 위험이 낮다.

자동화 결과가 GitHub/log 에 남는다.

30.2 자동화 단계

L0 Manual: 사람이 직접 수행.

L1 Checklist: 매번 같은 체크리스트로 수행.

L2 Script: deterministic script 로 수행.

L3 CI: GitHub Actions 에서 push/PR 마다 수행.

L4 Scheduled: 매일/매주 자동 실행.

L5 Event-triggered: issue/PR/source upload 이벤트에 반응.

L6 Agent-assisted: AI 가 초안/수정/분석을 수행하되 사람이 승인.

L7 Limited autonomy: 낮은 위험 작업만 자동 merge/route/archive.

L8 Full autonomy: Asperitas 에서는 당분간 금지. 특히 bio/legal/wet-lab/finance 외부발신은 금지.

30.3 우선 자동화 Top 20

1. metadata schema validation
2. source registry duplicate detection
3. file naming normalization
4. markdown extraction report
5. chunk count and checksum report
6. citation mismatch eval
7. golden query regression test
8. pytest on PR
9. artifact verification
10. release readiness checklist
11. dependency review
12. secret scan
13. license/SBOM report
14. PR template enforcement
15. decision log template generation
16. AGENTS.md repeated-feedback update proposal
17. daily source ingestion candidate brief
18. weekly eval failure report
19. monthly AI roadmap review
20. automation backlog prioritization

30.4 자동화 금지/보류 항목

- wet-lab protocol execution without human approval
- biosafety/biosecurity-sensitive design release

- legal/regulatory approval claim generation
 - investor/customer external communication send
 - production DB migration without backup and approval
 - restricted/private document upload to external tools
 - autonomous dependency upgrade in production branch
 - autonomous deletion of source/raw files
-

31. 강화해야 할 보완점과 금지 패턴

31.1 보완점 A - Eval 이 기능보다 먼저 와야 한다

다음부터는 “기능을 만든 뒤 테스트”가 아니라 “성공 기준을 먼저 고정한 뒤 구현”한다.

Before coding:

- What is the baseline?
- What can get worse?
- Which 10-50 examples prove improvement?
- What is the failure taxonomy?

31.2 보완점 B - 성능향상을 코드와 prompt 둘 다에서 본다

AI agent 성능은 한 레이어만 바뀌서는 안정적으로 좋아지지 않는다.

Prompt only improvement = brittle

Model only improvement = expensive

Data only improvement = incomplete

Code only improvement = narrow

Correct: Data + Retrieval + Prompt + Tool + Eval + UX + Governance

31.3 보완점 C - ChatGPT 사령탑을 유지한다

Codex/Claude 를 여러 채팅으로 분리해도 최상위 roadmap 과 decision log 는 하나여야 한다.

One command center.

Many execution threads.

Single source of truth.

GitHub-backed evidence.

31.4 보완점 D - Skills 를 적극 제품화한다

반복 작업은 다음 순서로 재사용 자산화한다.

repeated prompt -> template -> runbook -> script -> skill -> plugin/workflow

Asperitas 첫 Skills 후보:

source-ingestion-skill

eval-report-skill

release-readiness-skill

compliance-gate-skill

literature-claim-card-skill

github-pr-review-skill

decision-log-skill

31.5 금지 패턴

1. Prompt bloat

- 긴 지시를 계속 AGENTS.md 에 붙여 넣는 것.
- 해결: core rule + task-specific prompt + skill 분리.

2. Eval theater

- 실제 실패를 못 잡는 예쁜 점수표만 만드는 것.
- 해결: 실제 과거 실패 케이스를 golden set 에 넣기.

3. Context rot

- 한 채팅에서 모든 탐색/로그/구현/리뷰를 계속 이어가는 것.
- 해결: execution thread 분리 + decision summary 만 사령탑에 남기기.

4. Source laundering

- 외부 자료 요약을 내부 사실처럼 쓰는 것.
- 해결: source priority 와 evidence label 유지.

5. Automation before standardization

- SOP 도 없는 일을 agent 로 자동화하는 것.
- 해결: manual -> checklist -> script -> CI 순서.

6. Security blindness

- MCP/Skills/Hooks 를 편하다고 바로 연결하는 것.
- 해결: allowlist, sandbox, permissions, secret policy.

7. Bio overclaim

- 설계/문헌 근거를 실험 검증처럼 말하는 것.
- 해결: fact/inference/speculation/verification need 분리.

32. v3.2 Master Prompt for Integrated AI Engineering

Role:

You are the AI engineering lead for Asperitas AI Agent Factory. Operate as a product-minded, eval-driven, security-aware AI engineer, not as a passive code generator.

Context:

Asperitas is building a source-grounded, auditable, compliance-aware Biological Intelligence Factory. The current doctrine is AOS/PRIME + Agent Factory Playbook v3.2-RC1. Codex/Claude chats may be separated, but the process, acceptance criteria, and decision log must be defined from this command thread and preserved in GitHub.

Task:

For the next MVP, produce an integrated implementation plan that combines feature development and performance improvement in one package.

Required Process:

1. State the user/job-to-be-done.
2. Define current baseline and target metric.
3. Identify the engineering lever: data, retrieval, prompt, tool, model, schema, UI, eval, automation, or governance.
4. Define the smallest safe implementation diff.
5. Define test/eval/performance checks before coding.
6. Define GitHub branch/PR/release evidence.
7. Define what should become AGENTS.md rule, Skill, Runbook, or Automation.

Hard Rules:

- Do not implement without baseline and Definition of Done.
- Do not treat source map as production DB.
- Do not claim legal, regulatory, biosafety, wet-lab, or foundation model completion unless verified.
- Do not expose restricted data to external tools.
- Human approval is mandatory for high-risk bio, legal, investor, regulatory, IP, external communication, and wet-lab-sensitive outputs.

Output:

1. Executive Bottom Line
2. MVP Brief

3. Performance Pack
4. File/Module Plan
5. Test Plan
6. Eval Plan
7. Automation Plan
8. Security/Compliance Gate
9. GitHub Workflow
10. Decision Log Entry
11. Codex Prompt
12. Stop Conditions

Stop Rule:

If repo state, source status, metric, risk boundary, or approval requirement is unclear, stop and ask before coding.

33. v3.2 Official Learning Reference Anchors

Use these as external learning anchors, not as internal Asperitas facts.

OpenAI / Codex / Evals

- OpenAI Codex web/cloud: <https://developers.openai.com/codex/cloud>
- Codex AGENTS.md guide: <https://developers.openai.com/codex/guides/agents-md>
- Codex GitHub code review: <https://developers.openai.com/codex/integrations/github>
- Codex Skills: <https://developers.openai.com/codex/skills>
- Codex MCP: <https://developers.openai.com/codex/mcp>
- Codex Subagents: <https://developers.openai.com/codex/concepts/subagents>
- OpenAI Evals: <https://developers.openai.com/api/docs/guides/evals>
- OpenAI Agent Evals: <https://developers.openai.com/api/docs/guides/agent-evals>

Anthropic / Claude / MCP

- Agent Skills engineering post: <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>
- Anthropic agent evals: <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Claude Code hooks: <https://code.claude.com/docs/en/hooks-guide>
- MCP introduction: <https://modelcontextprotocol.io/docs/getting-started/intro>

Microsoft

- Microsoft Foundry docs: <https://learn.microsoft.com/en-us/azure/foundry/>

- What is Microsoft Foundry: <https://learn.microsoft.com/en-us/azure/foundry/what-is-foundry>
- Microsoft AI Learning Hub: <https://learn.microsoft.com/en-us/ai/>

MIT

- MIT Open Learning AI courses: <https://openlearning.mit.edu/news/explore-world-artificial-intelligence-online-courses-mit>
- MIT 6.S191 Introduction to Deep Learning: <https://introtodeeplearning.com/>

Stanford

- Stanford CS224N: <https://web.stanford.edu/class/cs224n/>
- Stanford CS231N: <https://cs231n.stanford.edu/>
- Stanford HAI AI Index: <https://hai.stanford.edu/ai-index/2026-ai-index-report>